

Design and Implementation of a Production-Ready CI/CD Pipeline on AWS

1. Project Overview

Project goals:

- Automate containerized deployments
- Improve deployment consistency
- Reduce infrastructure management overhead
- Follow secure AWS deployment practices

This project focused on building and deploying a complete CI/CD pipeline for a containerized full-stack application on AWS using Terraform, Jenkins, Amazon ECS (Fargate), and Amazon ECR.

The application consists of:

- React frontend
- Express backend

The main objective was to automate the entire deployment process from source code commit to production deployment while following secure and scalable cloud architecture practices.

The final solution allows developers to push code changes to GitHub, which automatically triggers a Jenkins pipeline to build Docker images, push them to Amazon ECR, and redeploy updated services on Amazon ECS.

This project combines infrastructure provisioning, container orchestration, CI/CD automation, networking, and cloud security into a single deployment workflow.

2. Architecture & Design Decisions

The architecture was designed with three priorities in mind:

- automation
- security
- scalability

Instead of deploying applications directly on EC2 instances, I used Amazon ECS with Fargate to remove infrastructure management overhead and simplify container orchestration.

Jenkins was deployed on an EC2 instance to provide full control over the CI/CD pipeline and deployment workflow.

Amazon ECR was used as the container registry for storing frontend and backend images, while Application Load Balancer handled external traffic routing.

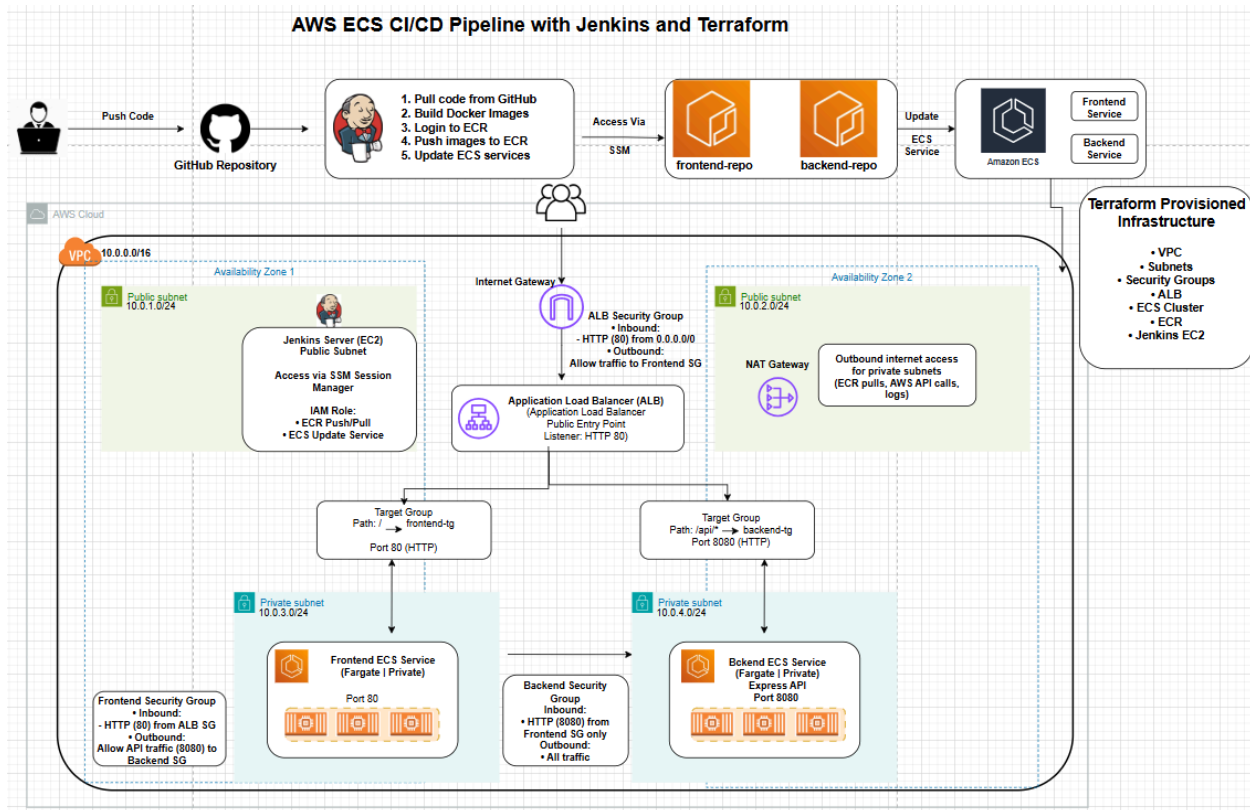
Traffic routing was configured as follows:

- `/` routes to frontend service
- `/api` routes to backend service

This allows both frontend and backend services to be accessed through a single public endpoint.

Architecture decisions made during implementation:

- Jenkins on EC2 for pipeline control and flexibility
- ECS Fargate to avoid managing container hosts
- Private subnets for application workloads
- ALB as the single public-facing component
- Session Manager instead of SSH for secure administration



One important design decision was keeping frontend and backend services inside private subnets. This ensures the application containers are not directly exposed to the internet.

Only the ALB remains publicly accessible.

3. Infrastructure Provisioning

Infrastructure provisioning was fully managed using Terraform.

The infrastructure included the following resources:

Networking

- VPC
- Public subnets
- Private subnets
- Internet Gateway

- NAT Gateway
- Route Tables

The VPC was configured to separate internet-facing resources from internal application resources.

Public subnets host:

- Application Load Balancer
- Jenkins EC2

Private subnets host:

- Frontend ECS tasks
- Backend ECS tasks

This separation improves security and reduces unnecessary exposure.

Compute

Compute resources included:

- Jenkins EC2 instance
- ECS Cluster
- ECS Services (frontend and backend)

Jenkins was responsible for CI/CD automation.

ECS Fargate handled application container execution.

Container Registry

Amazon ECR repositories were created for:

- frontend image
- backend image

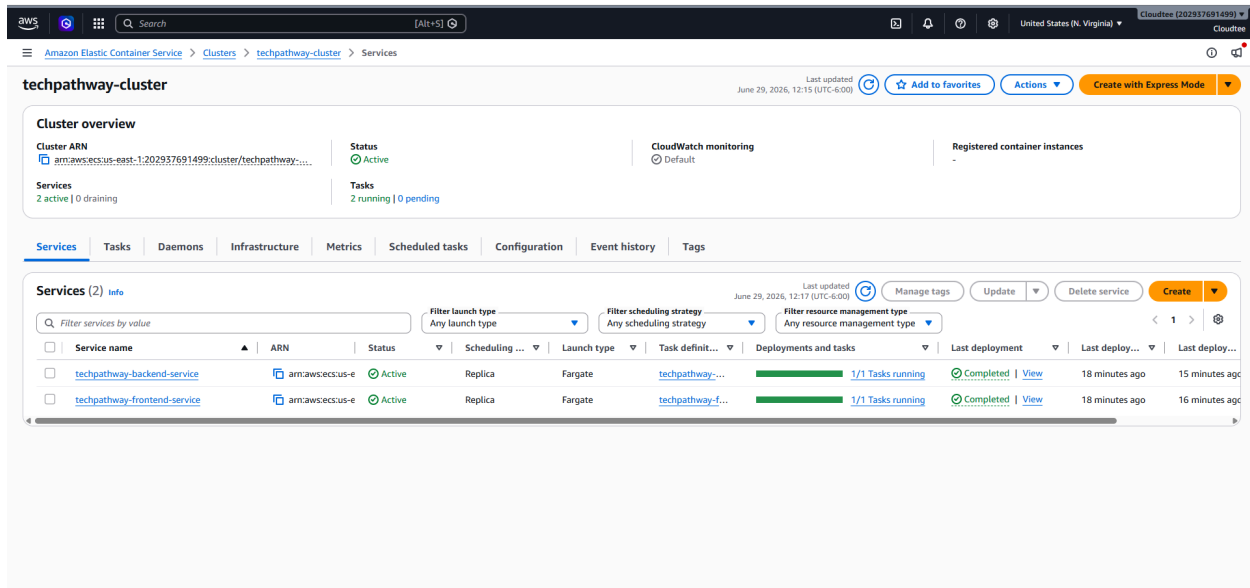
These repositories store container images built during Jenkins pipeline execution.

Load Balancing

Application Load Balancer was configured with:

- frontend target group
- backend target group
- listener rules

The ALB performs path-based routing to ensure traffic reaches the correct service.



4. Terraform Design & Deployment

Infrastructure provisioning was implemented using Terraform and organized into separate configuration files to improve maintainability and resource separation.

Terraform file structure included:

- provider.tf → AWS provider configuration
- variables.tf → Input variables
- networking.tf → VPC, subnets, route tables, internet gateway, NAT gateway
- security.tf → Security groups
- alb.tf → Application Load Balancer, target groups, listeners
- ecs.tf → ECS cluster, task definitions, services

- jenkins.tf → Jenkins EC2 infrastructure
- outputs.tf → Useful deployment outputs

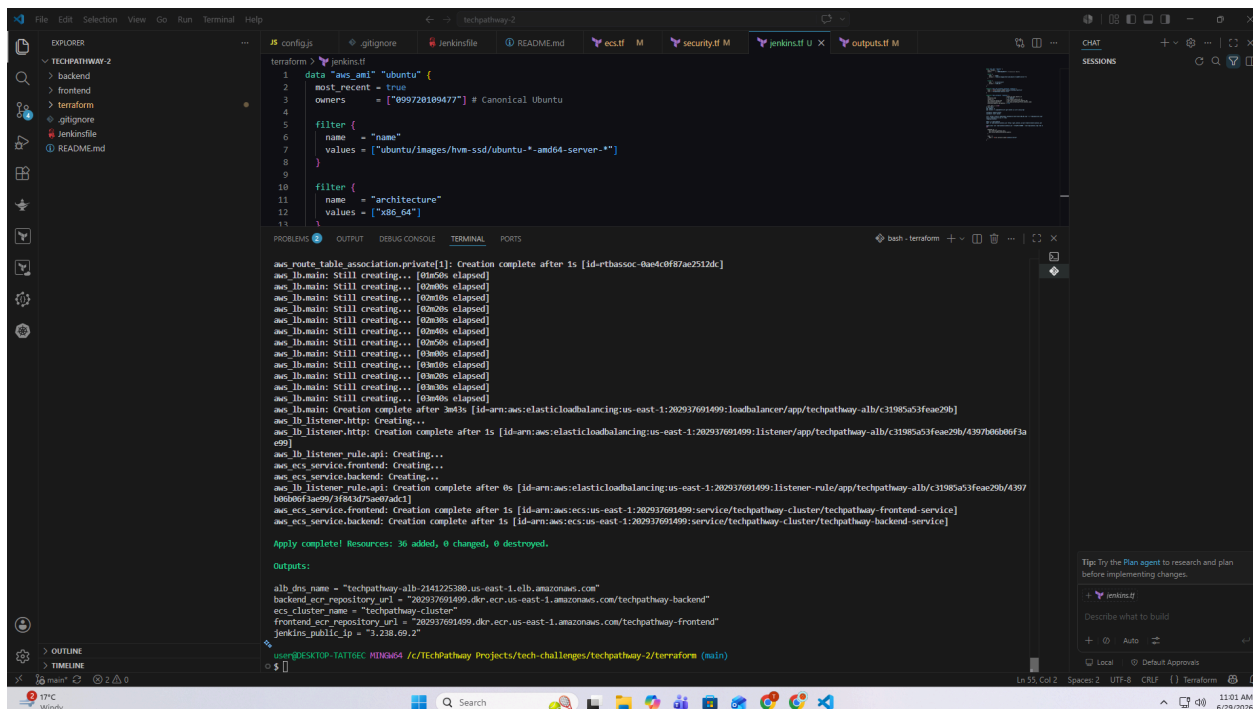
This structure improved readability and made infrastructure easier to manage.

Deployment Workflow

Infrastructure deployment followed a standard Terraform workflow:

```
terraform init
terraform plan
terraform apply
```

Terraform was responsible for provisioning all AWS infrastructure required for the application and CI/CD pipeline.



During project cleanup, infrastructure teardown was performed using:

```
terraform destroy
```

This ensured all cloud resources were removed cleanly after validation.

5. Security Design

Security was a major consideration throughout the project.

The architecture follows a layered security model.

Network Isolation

Frontend and backend ECS services run in private subnets without public IP addresses.

This ensures application workloads are only reachable through internal AWS networking.

Access Control

Application traffic flows through tightly controlled security groups.

Traffic rules:

- Internet → ALB
- ALB → Frontend Service
- Frontend → Backend Service

The backend service only accepts traffic from trusted internal sources.

This significantly reduces the attack surface.

Administrative Access

Jenkins administration does not rely on SSH.

Instead, AWS Systems Manager Session Manager was used for secure administrative access.

This eliminates the need for:

- open SSH ports
- key pair management
- public SSH exposure

This aligns with modern cloud security best practices.

IAM permissions were also scoped to least privilege where possible.

6. CI/CD Workflow

The CI/CD pipeline automates application delivery from development to deployment.

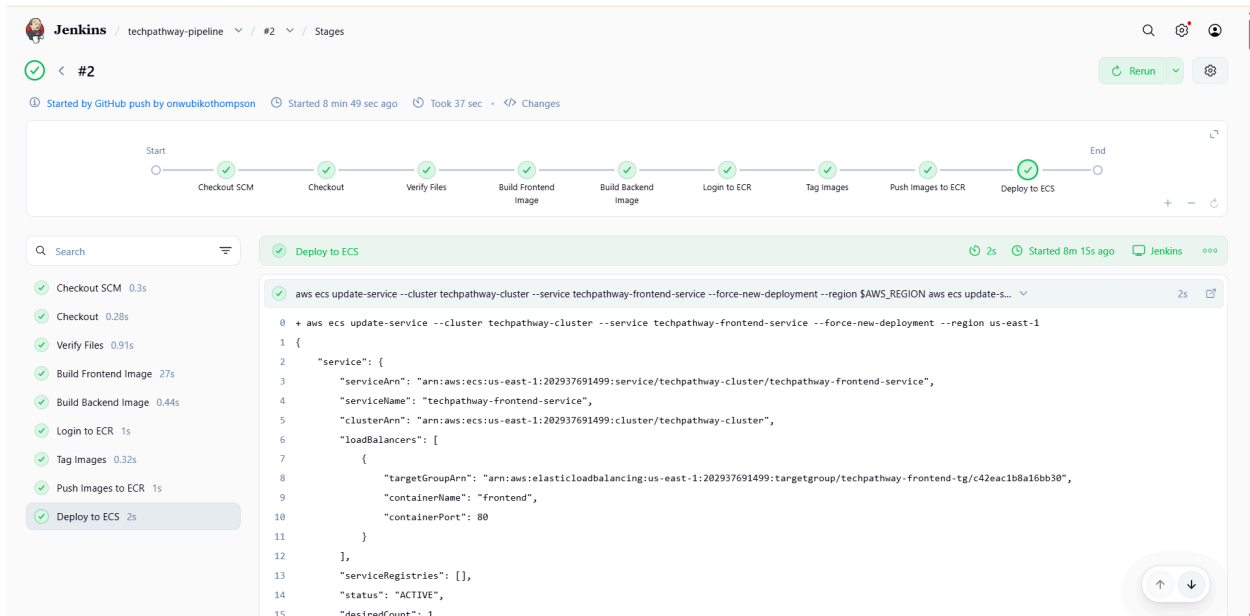
Pipeline workflow:

1. Developer pushes code to GitHub
2. GitHub webhook triggers Jenkins
3. Jenkins pulls latest source code
4. Frontend Docker image is built
5. Backend Docker image is built
6. Jenkins authenticates with Amazon ECR
7. Images are pushed to ECR
8. ECS services are updated
9. ECS pulls latest images and redeploys services

Pipeline stages implemented in Jenkins:

- Checkout SCM
- Verify Files
- Build Frontend Image
- Build Backend Image
- Login to ECR
- Tag Images
- Push Images to ECR
- Deploy to ECS

This pipeline removed manual deployment steps and improved deployment consistency.



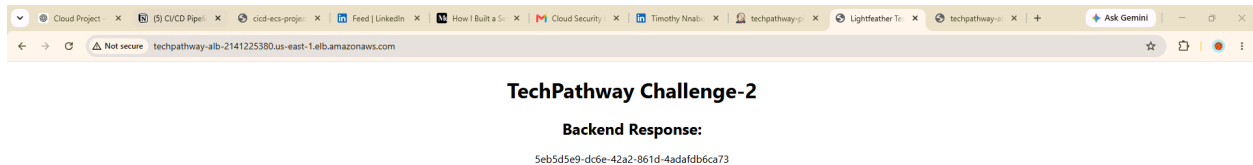
7. Validation & Troubleshooting

The environment was validated by testing the complete deployment workflow from GitHub push to production deployment.

Validation steps included:

- webhook trigger verification
- Jenkins pipeline validation
- ECR image push verification
- ECS deployment verification
- frontend accessibility testing
- backend API validation

The final deployment was verified successfully through the Application Load Balancer.



Several challenges came up during implementation.

GitHub Webhook Trigger Issues

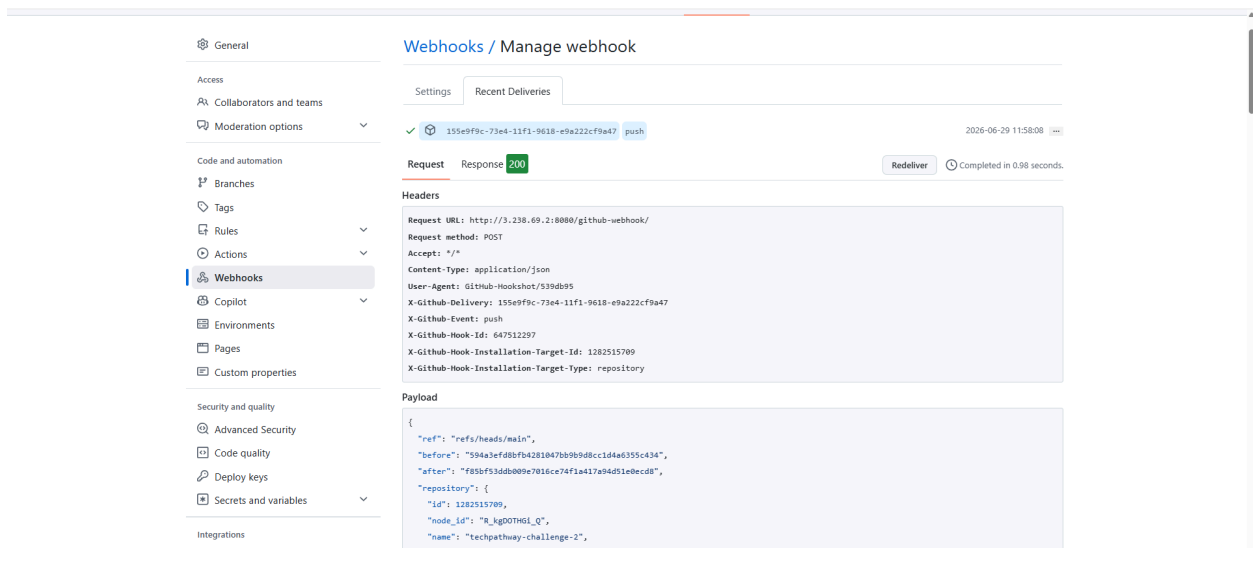
Initially, GitHub pushes were not triggering Jenkins builds.

Root cause:

Webhook configuration issues and trigger settings in Jenkins.

Resolution:

Webhook configuration was corrected and Jenkins GitHub trigger settings were updated.



Frontend Routing Issues

Frontend deployment succeeded, but API calls initially failed.

Root cause:

Frontend API endpoint configuration did not match ALB routing configuration.

Resolution:

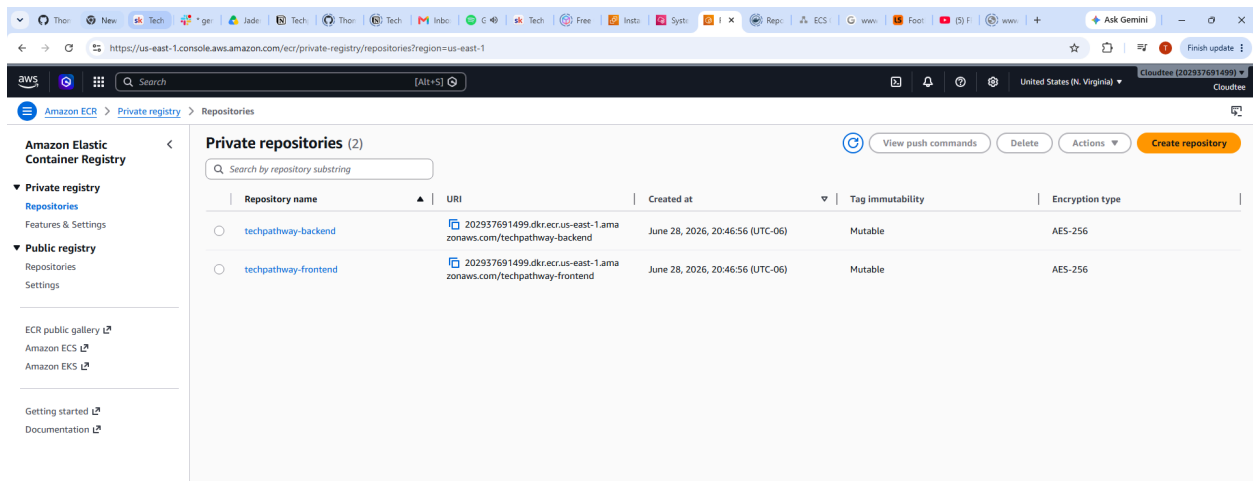
Frontend configuration was updated to use ALB-based API routing.

ECS Deployment Validation

Another challenge involved verifying whether ECS services were pulling the latest images.

This was resolved by:

- checking ECR image updates
- verifying ECS task redeployments
- validating service health



These troubleshooting steps helped improve confidence in the overall deployment workflow.

8. Future Improvements

Although the current implementation meets project goals, there are several improvements that would strengthen a production deployment.

Recommended improvements include:

- Centralized logging with CloudWatch
- Secrets management with AWS Secrets Manager
- Store Terraform state in S3 with DynamoDB locking
- Implement blue/green deployment strategy

These improvements would enhance scalability, resiliency, security, and operational visibility.

9. Lessons Learned

- CI/CD pipelines are only as reliable as their deployment validation strategy
- Secure networking design should be part of architecture from the beginning
- Infrastructure automation improves consistency and reduces manual errors

This makes it sound like a real engineering postmortem.

Final Remarks

This project reinforced practical experience in infrastructure automation, container orchestration, CI/CD pipeline design, and secure AWS deployments.

More importantly, it reinforced the importance of designing systems that are not only functional, but also secure, scalable, and operationally reliable.

The biggest takeaway from this project was understanding how infrastructure automation, CI/CD pipelines, and cloud-native services work together to deliver reliable application deployments.